

## Roh Machina

- **Had masa:** 10 minit
- **Skor baseline:** 28.6
- **Persekitaran:** satu GPU ( $\approx 16$  GB VRAM), tiada internet
- **Saiz penyelesaian:** `solution.ipynb`  $\leq 20$  MB
- **Storan:** 5 GB
- **Model pralatih:** hanya [bge-base-en-v1.5](#) — sebuah **encoder** teks (model embedding).

## Tugasan

Perkara-perkara pelik sedang berlaku di Arkib Negara Kazakhstan! Pustakawan-pustakawan berkata bahawa sebahagian buku sekarang mempunyai akhiran yang berbeza bebanding dahulu, tetapi tiada siapa dapat membuktikannya. Setiap salinan buku adalah sama, dan setiap cerita masih masuk akal lagi. Anda dijemput sebagai penyelidik AI untuk mengesan perubahan tersebut.



Sebuah petikan bermula sebagai teks tulisan manusia dan berakhir sebagai teks tulisan model bahasa. Pertukaran ini berlaku secara senyap. Apabila dibaca secara keseluruhan, petikan-petikan nampak seperti satu karya yang koheren, tetapi pada suatu tempat, penulisnya berubah daripada seorang manusia kepada sebuah mesin. Tugas anda ialah **mencari peralihan itu: indeks aksara (character) di mana bahagian manusia berakhir dan bahagian mesin bermula**.

Setiap sampel ialah satu rentetan (string) tunggal `text`. Terdapat satu sempadan perubahan sahaja. Segala-galanya sebelum perubahan tersebut ditulis manusia, selepas perubahan tersebut ditulis mesin.

## Set data

Set data adalah petikan-petikan bahasa Inggeris, dan setiap petikan mengandungi satu sempadan perubahan.

- **Bahagian A** (sebelum sempadan): satu cebisan teks tulisan manusia.
- **Bahagian B** (daripada sempadan dan seterusnya): satu sambungan yang dihasilkan oleh sebuah model bahasa, dijana dengan merujuk pada Bahagian A.
- Setiap belah mengandungi sekurang-kurangnya 180 perkataan; panjang keseluruhannya ~500–800 perkataan.
- **boundary\_char\_index** ialah ofset aksara (character) di mana Bahagian A berakhir: `text[:boundary_char_index]` ialah bahagian manusia dan `text[boundary_char_index:].rstrip()` ialah bahagian mesin.

### Apa yang anda terima

Anda menerima **dua folder**:

| Folder               | Sampel | answers.jsonl?             | Gunakan untuk                                                      |
|----------------------|--------|----------------------------|--------------------------------------------------------------------|
| dataset/train/       | 1,221  | ❑ disertakan               | melatih / fine-tune kaedah anda                                    |
| dataset/test_public/ | 380    | ❑ disertakan (salinan dev) | menjalankan pipeline anda dan menilai skor sendiri secara setempat |

Pada **masa penilaian**, folder `dataset/test_public/` anda akan **digantikan dengan satu set penilaian tersembunyi**. Ia mempunyai format yang sama tetapi **tanpa answers.jsonl**. Notebook anda dijalankan semula padanya, dan `answers.jsonl` yang dihasilkannya akan diberi skor.

- Papan pendahulu awam menggunakan satu set tersembunyi **test\_leaderboard\_a** (380 sampel).
- Kedudukan akhir menggunakan satu set tersembunyi **test\_leaderboard\_b** (380 sampel).

Ketiga-tiga set penilaian mempunyai saiz yang sama dan diambil daripada taburan (distribution) yang sama seperti `train`, jadi skor `dataset/test_public/` setempat anda merupakan penganggaran yang berpatutan bagi skor papan pendahulu anda.

### Format fail

```
dataset/train/data.jsonl      # one JSON object per line: {"id": "...", "text": "..."}
dataset/train/answers.jsonl  # {"id": "...", "boundary_char_index": 1842}
dataset/test_public/data.jsonl      # {"id": "...", "text": "..."}
dataset/test_public/answers.jsonl  # dev copy only – ABSENT in the hidden grading set
```

- id dalam `answers.jsonl` sepadan dengan id dalam `data.jsonl`.
- `dataset/train/` (dengan jawapan) tersedia bila-bila masa anda melatih atau melakukan fine-tune.

## Output (format penyerahan)

Anda menyerahkan **satu notebook tunggal, yang mesti dinamakan solution.ipynb**. Nama fail tepat ini adalah wajib. Apa-apa nama selain daripadanya akan ditolak secara langsung.

Notebook anda mesti **membaca** `dataset/test_public/data.jsonl` dan menulis satu fail tunggal `answers.jsonl` pada akar repositori. Fail tersebut harus mengandungi satu objek JSON setiap baris, dan setiap objek JSON harus merujukkan setiap id sampel kepada indeks aksara sempadan yang anda ramalkan:

```
{"id": "example_000123", "boundary_char_index": 1790}
{"id": "example_000124", "boundary_char_index": 2450}
```

- `boundary_char_index` mesti merupakan satu **integer dalam** `[0, len(text)]`.
- Setiap id dalam `dataset/test_public/data.jsonl` patut muncul sekali sahaja. Satu sampel yang tiada dalam `answers.jsonl` (atau dengan nilai bukan integer / di luar julat) mendapat skor 0 bagi sampel tersebut.

## Pemberian skor

Bagi setiap sampel, biarlah  $p$  sebagai indeks yang anda ramalkan dan  $t$  sebagai sempadan yang benar. Skor bagi setiap sampel menjatuh secara eksponen dengan jarak aksara:

$$\text{score} = \exp\left(-\frac{|p - t|}{\tau}\right) \in (0, 1], \text{ where } \tau = 100.$$

Ini membawa kepada kelakuan skor yang berikut:

- **=1.0**: aksara sempadan yang tepat;
- **≈0.78**: tersasar 25 aksara; - **≈0.61** — tersasar 50 aksara;
- **≈0.37**: tersasar 100 aksara;
- **≈0.01**: tersasar 500 aksara.

**Skor akhir adalah purata** bagi skor setiap sampel merentas semua sampel dalam split tersebut (dilaporkan pada skala 0–100). Metrik ini memberi ganjaran kepada ramalan yang *hampir*, bukan sekadar yang tepat.

## Kekangan

- **Persekitaran**: satu GPU (≈16 GB VRAM), tiada internet pada masa penilaian; model yang dibenarkan (di bawah) sudah pun disediakan. **Bajet masa wall-clock: 10 minit** untuk keseluruhan larian; ini mesti merangkumi apa-apa latihan / fine-tuning yang anda lakukan pada masa penilaian **serta** inferens pada set penilaian.
- **Model pralatih yang dibenarkan**: hanya model pralatih berikut dibenarkan. Ia **disediakan terlebih dahulu dalam persekitaran** (muatkannya seperti biasa, cth. `from_pretrained`; tiada internet pada masa penilaian):
  - **bge-base-en-v1.5** : sebuah **encoder** teks 110M parameter (model embedding). Ia menghasilkan embedding ayat/petikan; ia bukan model bahasa generatif. Anda boleh menggunakannya **sebagaimana adanya (ciri yang dibekukan) atau melakukan fine-tune padanya pada split train** (fine-tuning penuh sesuai dengan bajet 16 GB / 10 minit).
- Cara-cara klasik / statistik adalah dibenarkan tanpa batasan: anda boleh membina apa-apa model berasaskan ciri (cth., pengelas atau peng-regres scikit-learn) di atas ciri embedding yang anda membina sendiri. *Pemberat pembelajaran mendalam pralatih* sahaja yang dihadkan kepada senarai di atas.